

Chapter 17 — Fast and Slow Pointers

0. What You Will Learn in This Chapter

Fast and Slow Pointers is a specialized linked-list and sequence pattern. It is often called the **tortoise and hare** technique.

The idea is simple:

- one pointer moves slowly
- another pointer moves faster

This pattern is used to detect cycles, find middle positions, locate cycle starts, and solve problems where different speeds reveal structure.

By the end of this chapter, you should understand:

- What fast and slow pointers are
- Why different speeds help
- When to use this pattern
- When not to use it
- How Floyd's Cycle Detection works
- How to find the start of a cycle
- How to find the middle of a linked list
- How to remove nth node from end using pointer gap
- How to detect duplicate number using cycle logic
- How Happy Number becomes a cycle problem
- How to solve palindrome linked list
- How to split linked lists safely
- Common edge cases
- Common null-pointer mistakes
- How to explain this pattern in interviews

Core problems covered in this chapter:

1. Middle of the Linked List
 2. Linked List Cycle
 3. Linked List Cycle II
 4. Happy Number
 5. Find the Duplicate Number
 6. Remove Nth Node From End of List
 7. Palindrome Linked List
 8. Reorder List
 9. Split Linked List in Parts
 10. Circular Array Loop
 11. Detect Cycle in Directed Functional Graph
 12. Find the Winner of the Circular Game, concept contrast
-

1. What Are Fast and Slow Pointers?

Fast and slow pointers are two references moving through a structure at different speeds.

Most common linked-list setup:

```
ListNode* slow = head;
ListNode* fast = head;

while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
}
```

Here:

- slow moves one step
- fast moves two steps

When fast reaches the end, slow is near the middle.

If there is a cycle, fast eventually catches slow.

2. Why Fast and Slow Pointers Exist

Some linked-list problems cannot use indexing.

For arrays, finding middle is easy:

```
mid = n / 2;
```

For linked lists, there is no direct indexing.

You could count nodes first, then traverse again.

But fast/slow pointers do it in one pass.

Example:

```
1 -> 2 -> 3 -> 4 -> 5
```

Move:

- slow by 1
- fast by 2

When fast reaches end, slow reaches 3.

3. Core Intuition

Fast pointer reveals global progress.

Slow pointer marks the corresponding halfway or meeting point.

For cycle detection:

- if no cycle, fast reaches nullptr
- if cycle exists, fast keeps looping and eventually catches slow

Why?

Inside the cycle, fast gains one node per iteration on slow.

Eventually the distance between them becomes zero.

4. Core Need / Why Use Fast-Slow Over Other Patterns?

4.1 Why Not Count Length?

Counting length works for middle, but requires two passes.

Fast-slow can find middle in one pass.

4.2 Why Not Hash Set for Cycle?

Hash set can detect repeated nodes:

```
unordered_set<ListNode*> seen;
```

But it uses $O(n)$ extra space.

Fast-slow detects cycles in $O(1)$ space.

4.3 Why Not Array Conversion?

You can copy linked list values into vector, but that uses $O(n)$ space.

Fast-slow is better when $O(1)$ space is requested.

4.4 Why Not Binary Search?

Linked lists do not support $O(1)$ middle access.

Binary search is inefficient on linked lists.

5. When to Use Fast and Slow Pointers

Use this pattern when the problem asks for:

5.1 Middle of Linked List

Examples:

- Middle of the Linked List
- Sort List splitting
- Reorder List

5.2 Cycle Detection

Examples:

- Linked List Cycle
- Linked List Cycle II
- Happy Number
- Find Duplicate Number

5.3 Cycle Start

If cycle exists and you need entry point.

Example:

- Linked List Cycle II

5.4 Two Pointers With Gap

This is a related linked-list pointer technique.

Examples:

- Remove Nth Node From End
- Find kth node from end

5.5 Split / Reverse / Compare

Examples:

- Palindrome Linked List
 - Reorder List
-

6. When NOT to Use Fast and Slow Pointers

6.1 Need Sorted Relationship

Use sorting, binary search, or two pointers depending on problem.

6.2 Need Arbitrary Lookup

Use hash map/set.

6.3 Need BFS/Shortest Path

Use queue.

6.4 Need Range Queries

Use prefix sum, Fenwick tree, or segment tree.

6.5 Need All Combinations

Use backtracking.

Fast/slow is for structural traversal, not enumeration.

7. Mental Models

7.1 Race Track

If two runners are on a circular track and one is faster, the faster runner eventually catches the slower runner.

This explains cycle detection.

7.2 Clock Hands

Fast hand moves faster than slow hand.

If movement is circular, they meet again.

7.3 Measuring Half Without Length

Fast takes two steps for every one step slow takes.

So when fast finishes, slow has traveled half the distance.

7.4 Gap Pointer

For nth-from-end:

Keep a fixed gap between fast and slow.

When fast reaches end, slow is positioned correctly.

8. Pattern Recognition Checklist

Ask:

13. Is this a linked list?
14. Do I need the middle node?
15. Do I need to detect a cycle?
16. Do I need to find cycle entry?
17. Do I need $O(1)$ space?
18. Can one pointer move faster?
19. Does the process eventually enter a cycle?
20. Can values be treated as next indices?
21. Do I need nth node from the end?
22. Is a two-pointer gap useful?

If yes, fast-slow pointers may apply.

9. C++ Tools Needed

9.1 Null-Safe Loop

Most common:

```
while (fast != nullptr && fast->next != nullptr) {  
    slow = slow->next;  
    fast = fast->next->next;  
}
```

This prevents accessing:

```
fast->next
```

when fast is null.

9.2 Cycle Meeting Check

```
if (slow == fast) {  
    return true;  
}
```

Compare node pointers, not values.

9.3 Reset Pointer to Head

For cycle entry:

```
ListNode* entry = head;  
  
while (entry != slow) {  
    entry = entry->next;  
    slow = slow->next;  
}
```

10. Core Templates

Template 1 — Find Middle

```
ListNode* middleNode(ListNode* head) {  
    ListNode* slow = head;  
    ListNode* fast = head;  
  
    while (fast != nullptr && fast->next != nullptr) {  
        slow = slow->next;  
        fast = fast->next->next;  
    }
```

```
    return slow;
}
```

Returns second middle for even length.

Template 2 — Detect Cycle

```
bool hasCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast) {
            return true;
        }
    }

    return false;
}
```

Template 3 — Find Cycle Entry

```
ListNode* detectCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast) {
            ListNode* entry = head;

            while (entry != slow) {
                entry = entry->next;
                slow = slow->next;
            }

            return entry;
        }
    }
}
```

```
    return nullptr;
}
```

Template 4 — Remove Nth From End

```
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0);
    dummy.next = head;

    ListNode* fast = &dummy;
    ListNode* slow = &dummy;

    for (int i = 0; i < n; i++) {
        fast = fast->next;
    }

    while (fast->next != nullptr) {
        fast = fast->next;
        slow = slow->next;
    }

    slow->next = slow->next->next;

    return dummy.next;
}
```

Template 5 — Find Duplicate Number by Cycle

```
int findDuplicate(vector<int>& nums) {
    int slow = nums[0];
    int fast = nums[0];

    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while (slow != fast);

    int entry = nums[0];

    while (entry != slow) {
        entry = nums[entry];
        slow = nums[slow];
    }
```

```

    }

    return entry;
}

```

11. Major Variations

Middle finding	fast moves 2x	Middle of List
Cycle exists	fast catches slow	Linked List Cycle
Cycle entry	reset one pointer	Linked List Cycle II
Value-index cycle	nums[i] as next pointer	Find Duplicate Number
Number transformation cycle	repeated function eventually loops	Happy Number
Pointer gap	fixed distance	Remove Nth From End
Reverse second half	middle + reverse	Palindrome Linked List
Split and merge	middle + reverse + weave	Reorder List

12. Full Problem Walkthrough 1 — LeetCode 876: Middle of the Linked List

12.1 Problem Summary

Given head of linked list, return middle node.

If there are two middle nodes, return the second middle.

Example:

1 -> 2 -> 3 -> 4 -> 5

Output:

3

Example:

1 -> 2 -> 3 -> 4 -> 5 -> 6

Output:

12.2 How to Identify This Pattern

Need middle of linked list.

Linked list has no index access.

Use fast and slow pointers.

12.3 Brute Force Idea

First pass: count nodes.

Second pass: move length / 2 steps.

Time $O(n)$, but two passes.

12.4 Optimized Intuition

Move:

- slow one step
- fast two steps

When fast reaches end, slow is middle.

12.5 C++ Code

```
class Solution {
public:
    ListNode* middleNode(ListNode* head) {
        ListNode* slow = head;
        ListNode* fast = head;

        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;
        }

        return slow;
    }
};
```

12.6 Dry Run

Input:

1 -> 2 -> 3 -> 4 -> 5

1	1
2	3

slow	fast
3	5

Fast cannot move further.

Return slow = 3.

12.7 Complexity

Time:

$O(n)$

Space:

$O(1)$

12.8 Edge Cases

- one node
 - two nodes
 - odd length
 - even length
-

12.9 Common Mistake

Using a loop condition that returns first middle instead of second middle when problem asks for second.

This template returns second middle for even length.

13. Full Problem Walkthrough 2 — LeetCode 141: Linked List Cycle

13.1 Problem Summary

Given a linked list, return true if it has a cycle.

13.2 How to Identify This Pattern

Need cycle detection in $O(1)$ space.

Use Floyd's Tortoise and Hare.

13.3 Brute Force Idea

Store visited nodes in hash set.

If a node repeats, cycle exists.

Time:

$O(n)$

Space:

$O(n)$

13.4 Optimized Intuition

Use two pointers.

If no cycle:

fast reaches nullptr

If cycle exists:

fast eventually catches slow

13.5 C++ Code

```
class Solution {
public:
    bool hasCycle(ListNode* head) {
        ListNode* slow = head;
        ListNode* fast = head;

        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;

            if (slow == fast) {
                return true;
            }
        }

        return false;
    }
};
```

13.6 Correctness Explanation

Inside a cycle, fast gains one node on slow each iteration. Because cycle length is finite, this relative distance eventually becomes zero, so they meet.

13.7 Complexity

Time:

$O(n)$

Space:

$O(1)$

13.8 Common Mistakes

- Comparing node values instead of pointers
- Not checking `fast != nullptr`

- Starting fast->next->next without null checks
-

14. Full Problem Walkthrough 3 — LeetCode 142: Linked List Cycle II

14.1 Problem Summary

Given linked list, return node where cycle begins.

If no cycle, return nullptr.

14.2 How to Identify This Pattern

Need not only detect cycle, but find cycle entry.

Use Floyd's algorithm:

23. Find meeting point.
 24. Reset one pointer to head.
 25. Move both one step.
 26. They meet at cycle entry.
-

14.3 Why Reset Works

Let:

- distance from head to cycle entry = a
- distance from cycle entry to meeting point = b
- remaining cycle length from meeting point back to entry = c

Slow travels:

a + b

Fast travels:

a + b + c + b

Fast travels twice slow:

$2(a + b) = a + 2b + c$

So:

a = c

modulo cycle length.

Therefore, one pointer from head and one from meeting point meet at entry.

14.4 C++ Code

```
class Solution {  
public:  
    ListNode* detectCycle(ListNode* head) {  
        ListNode* slow = head;  
        ListNode* fast = head;
```

```
while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;

    if (slow == fast) {
        ListNode* entry = head;

        while (entry != slow) {
            entry = entry->next;
            slow = slow->next;
        }

        return entry;
    }
}

return nullptr;
};
```

14.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

14.6 Edge Cases

- no cycle
- cycle starts at head
- one node with cycle to itself
- long non-cycle prefix

15. Full Problem Walkthrough 4 — LeetCode 202: Happy Number

15.1 Problem Summary

A happy number repeatedly replaces itself by sum of squares of digits.

If it reaches 1, it is happy.

If it loops forever, it is not happy.

Example:

19

Process:

19 -> 82 -> 68 -> 100 -> 1

Output:

true

15.2 How to Identify This Pattern

Repeated transformation eventually either:

- reaches 1
- enters a cycle

This is cycle detection.

Can use hash set or fast-slow pointers.

15.3 C++ Code

```
class Solution {
public:
    int nextNumber(int n) {
        int sum = 0;

        while (n > 0) {
            int digit = n % 10;
            sum += digit * digit;
            n /= 10;
        }

        return sum;
    }

    bool isHappy(int n) {
        int slow = n;
        int fast = nextNumber(n);

        while (fast != 1 && slow != fast) {
            slow = nextNumber(slow);
            fast = nextNumber(nextNumber(fast));
        }

        return fast == 1;
    }
};
```

15.4 Complexity

Time:

$O(\log n)$

per transformation, and the sequence becomes bounded.

Space:

$O(1)$

15.5 Similar Problems

- Linked List Cycle
- Find Duplicate Number
- Functional graph cycle detection

16. Full Problem Walkthrough 5 — LeetCode 287: Find the Duplicate Number

16.1 Problem Summary

Given array `nums` with $n + 1$ integers where each integer is in range $1..n$.

There is exactly one repeated number.

Return the duplicate.

Constraints:

- Do not modify array
- Use $O(1)$ extra space

Example:

`nums = [1,3,4,2,2]`

Output:

2

16.2 How to Identify Fast-Slow Pattern

Treat array as linked list:

`index -> nums[index]`

Since values are in $1..n$, each value points to another index.

Duplicate creates a cycle.

16.3 C++ Code

```
class Solution {
public:
    int findDuplicate(vector<int>& nums) {
        int slow = nums[0];
        int fast = nums[0];
```

```
do {
    slow = nums[slow];
    fast = nums[nums[fast]];
} while (slow != fast);

int entry = nums[0];

while (entry != slow) {
    entry = nums[entry];
    slow = nums[slow];
}

return entry;
}
```

16.4 Dry Run Concept

For:

nums = [1,3,4,2,2]

Pointers:

0 -> 1 -> 3 -> 2 -> 4 -> 2 ...

Cycle starts at value 2.

Return 2.

16.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

16.6 Common Mistakes

- Treating values as arbitrary; they are valid next indices because values are 1..n
- Starting incorrectly
- Returning meeting point without finding entry
- Modifying array when not allowed

17. Full Problem Walkthrough 6 — LeetCode 19: Remove Nth Node From End of List

17.1 Problem Summary

Remove nth node from end of list.

Example:

```
1 -> 2 -> 3 -> 4 -> 5
n = 2
```

Output:

```
1 -> 2 -> 3 -> 5
```

17.2 How to Identify This Pattern

Need node based on distance from end.

Use two pointers with fixed gap.

This is not fast/slow by speed, but it is the same two-pointer linked-list family.

17.3 C++ Code

```
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode dummy(0);
        dummy.next = head;

        ListNode* fast = &dummy;
        ListNode* slow = &dummy;

        for (int i = 0; i < n; i++) {
            fast = fast->next;
        }

        while (fast->next != nullptr) {
            fast = fast->next;
            slow = slow->next;
        }

        slow->next = slow->next->next;

        return dummy.next;
    }
};
```

```
}  
};
```

17.4 Complexity

Time:

$O(n)$

Space:

$O(1)$

17.5 Edge Cases

- remove head
- remove tail
- one node
- n equals length

18. Full Problem Walkthrough 7 — LeetCode 234: Palindrome Linked List

18.1 Problem Summary

Return true if linked list values form a palindrome.

Example:

1 -> 2 -> 2 -> 1

Output:

true

18.2 How to Identify This Pattern

Need compare first half with reversed second half.

Use:

27. fast/slow to find middle
28. reverse second half
29. compare

18.3 C++ Code

```
class Solution {  
public:  
    ListNode* reverse(ListNode* head) {  
        ListNode* prev = nullptr;  
        ListNode* curr = head;
```

```

while (curr != nullptr) {
    ListNode* nextNode = curr->next;
    curr->next = prev;
    prev = curr;
    curr = nextNode;
}

return prev;
}

bool isPalindrome(ListNode* head) {
    if (head == nullptr || head->next == nullptr) {
        return true;
    }

    ListNode* slow = head;
    ListNode* fast = head;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }

    ListNode* second = reverse(slow);
    ListNode* first = head;

    while (second != nullptr) {
        if (first->val != second->val) {
            return false;
        }

        first = first->next;
        second = second->next;
    }

    return true;
}
};

```

18.4 Complexity

Time:

$O(n)$

Space:

$O(1)$

18.5 Common Mistakes

- Comparing pointers instead of values
- Failing odd-length lists
- Losing second half during reversal

19. Full Problem Walkthrough 8 — LeetCode 143: Reorder List

19.1 Problem Summary

Reorder:

$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$

to:

$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \dots$

19.2 How to Identify This Pattern

Need middle, reverse second half, merge.

This combines:

- fast/slow
- reverse linked list
- merge alternating

19.3 C++ Code

```
class Solution {
public:
    ListNode* reverse(ListNode* head) {
        ListNode* prev = nullptr;
        ListNode* curr = head;

        while (curr != nullptr) {
            ListNode* nextNode = curr->next;
            curr->next = prev;
            prev = curr;
            curr = nextNode;
        }

        return prev;
    }
};
```

```

}

void reorderList(ListNode* head) {
    if (head == nullptr || head->next == nullptr) {
        return;
    }

    ListNode* slow = head;
    ListNode* fast = head;

    while (fast->next != nullptr && fast->next->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }

    ListNode* second = slow->next;
    slow->next = nullptr;

    second = reverse(second);

    ListNode* first = head;

    while (second != nullptr) {
        ListNode* temp1 = first->next;
        ListNode* temp2 = second->next;

        first->next = second;
        second->next = temp1;

        first = temp1;
        second = temp2;
    }
}
};

```

19.4 Complexity

Time:

$O(n)$

Space:

$O(1)$

20. Full Problem Walkthrough 9 — LeetCode 725: Split Linked List in Parts

20.1 Problem Summary

Split linked list into k consecutive parts as evenly as possible.

Earlier parts should have size greater than or equal to later parts.

Example:

```
head = 1 -> 2 -> 3  
k = 5
```

Output:

```
[1], [2], [3], [], []
```

20.2 How to Identify This Pattern

Need list length and controlled splitting.

This is linked-list traversal and pointer breaking.

20.3 C++ Code

```
class Solution {  
public:  
    vector<ListNode*> splitListToParts(ListNode* head, int k) {  
        int length = 0;  
        ListNode* curr = head;  
  
        while (curr != nullptr) {  
            length++;  
            curr = curr->next;  
        }  
  
        int baseSize = length / k;  
        int extra = length % k;  
  
        vector<ListNode*> result;  
  
        curr = head;  
  
        for (int i = 0; i < k; i++) {  
            ListNode* partHead = curr;  
            int partSize = baseSize + (i < extra ? 1 : 0);  
            for (int j = 1; j < partSize; j++) {  
                curr = curr->next;  
            }  
            result.push_back(partHead);  
            if (curr) {  
                curr->next = nullptr;  
            }  
        }  
        return result;  
    }  
};
```

```

        for (int j = 0; j < partSize - 1; j++) {
            if (curr != nullptr) {
                curr = curr->next;
            }
        }

        if (curr != nullptr) {
            ListNode* nextPart = curr->next;
            curr->next = nullptr;
            curr = nextPart;
        }

        result.push_back(partHead);
    }

    return result;
}
};

```

20.4 Complexity

Time:

$O(n + k)$

Space:

$O(k)$

for result array.

21. Full Problem Walkthrough 10 — LeetCode 457: Circular Array Loop

21.1 Problem Summary

Given circular array of non-zero integers.

Each value tells how many steps to move.

Return true if there is a cycle with:

- length greater than 1
- all movements in same direction

21.2 How to Identify Fast-Slow Pattern

This is a functional graph:

index -> next index

Need detect cycle under constraints.
Use fast/slow pointers over indices.

21.3 C++ Code

```
class Solution {
public:
    int nextIndex(vector<int>& nums, int i) {
        int n = nums.size();
        return ((i + nums[i]) % n + n) % n;
    }

    bool circularArrayLoop(vector<int>& nums) {
        int n = nums.size();

        for (int i = 0; i < n; i++) {
            int slow = i;
            int fast = i;
            bool direction = nums[i] > 0;

            while (true) {
                int nextSlow = nextIndex(nums, slow);

                if ((nums[nextSlow] > 0) != direction) {
                    break;
                }

                int nextFast = nextIndex(nums, fast);

                if ((nums[nextFast] > 0) != direction) {
                    break;
                }

                int nextFast2 = nextIndex(nums, nextFast);

                if ((nums[nextFast2] > 0) != direction) {
                    break;
                }

                slow = nextSlow;
                fast = nextFast2;

                if (slow == fast) {
```

```

        if (slow == nextIndex(nums, slow)) {
            break;
        }

        return true;
    }
}
}

return false;
}
};

```

21.4 Complexity

Basic version:

$O(n^2)$

Can be optimized by marking visited paths.
This version is easier for understanding.

21.5 Common Mistakes

- Allowing one-element cycle
- Mixing positive and negative directions
- Wrong modulo for negative movement

22. Mini Problem 1 — First Middle vs Second Middle

For even list:

1 -> 2 -> 3 -> 4

This loop returns second middle 3:

```

slow = head;
fast = head;

while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
}

```

This loop helps stop at first middle 2:

```

slow = head;
fast = head;

```

```
while (fast->next != nullptr && fast->next->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
}
```

Used for splitting lists.

23. Mini Problem 2 — Kth Node From End

```
ListNode* kthFromEnd(ListNode* head, int k) {
    ListNode* fast = head;
    ListNode* slow = head;

    for (int i = 0; i < k; i++) {
        fast = fast->next;
    }

    while (fast != nullptr) {
        fast = fast->next;
        slow = slow->next;
    }

    return slow;
}
```

Assumes k is valid.

24. Mini Problem 3 — Cycle Length

After detecting meeting point:

```
int cycleLength(ListNode* meet) {
    int length = 1;
    ListNode* curr = meet->next;

    while (curr != meet) {
        length++;
        curr = curr->next;
    }
}
```

```
    return length;
}
```

25. Mini Problem 4 — Functional Graph Cycle Detection

If every index points to exactly one next index:

```
next = f(current)
```

Then fast-slow can detect cycles.

Examples:

- Happy Number
- Find Duplicate Number
- Circular Array Loop

26. Common Edge Cases

26.1 Empty List

Return nullptr or false depending on problem.

26.2 One Node

Middle is itself.

Cycle exists only if `head->next == head`.

26.3 Two Nodes

Important for middle and cycle checks.

26.4 No Cycle

Fast reaches null.

26.5 Cycle Starts at Head

Cycle entry logic still works.

26.6 Even Length Middle

Know whether problem wants first or second middle.

26.7 Null Checks

Always check:

```
fast != nullptr && fast->next != nullptr
```

before moving fast two steps.

27. Common Mistakes and Wrong Code

Mistake 1 — Comparing Values Instead of Pointers

Wrong:

```
if (slow->val == fast->val)
```

Correct:

```
if (slow == fast)
```

Mistake 2 — Missing Null Check

Wrong:

```
while (fast->next != nullptr) {  
    fast = fast->next->next;  
}
```

Correct:

```
while (fast != nullptr && fast->next != nullptr) {  
    fast = fast->next->next;  
}
```

Mistake 3 — Returning Meeting Point as Cycle Entry

Meeting point is not necessarily cycle start.

Need reset pointer to head.

Mistake 4 — Wrong Middle for Split

For reorder/sort, sometimes you need first middle to split correctly.

Mistake 5 — Not Using Dummy for Remove Nth

Removing head becomes difficult without dummy.

28. Interview Explanation Templates

28.1 Middle of Linked List

I use two pointers. The slow pointer moves one step and the fast pointer moves two steps. When fast reaches the end, slow has moved half the distance, so it points to the middle.

28.2 Cycle Detection

If there is no cycle, the fast pointer reaches null. If there is a cycle, fast moves around the cycle faster than slow and eventually catches it.

28.3 Cycle Entry

After slow and fast meet, I reset one pointer to head. Moving both one step at a time makes them meet at the cycle entry due to the distance relationship between the head, entry, and meeting point.

28.4 Remove Nth From End

I maintain a gap of n nodes between fast and slow. When fast reaches the end, slow is right before the node that must be removed.

28.5 Find Duplicate Number

I model the array as a linked list where index points to nums[index]. The duplicate creates a cycle, and the cycle entry is the duplicate number.

29. Complexity Patterns

Middle of list	$O(n)$	$O(1)$
Cycle detection	$O(n)$	$O(1)$
Cycle entry	$O(n)$	$O(1)$
Happy Number	bounded	$O(1)$
Find Duplicate Number	$O(n)$	$O(1)$
Remove Nth From End	$O(n)$	$O(1)$
Palindrome List	$O(n)$	$O(1)$
Reorder List	$O(n)$	$O(1)$
Split List in Parts	$O(n + k)$	$O(k)$
Circular Array Loop	$O(n^2)$ basic	$O(1)$

30. Time-Boxed Practice Guidance

Easy Fast-Slow Problems

Target:

10–20 minutes

Examples:

- Middle of the Linked List
- Linked List Cycle
- Happy Number

Medium Fast-Slow Problems

Target:

20–40 minutes

Examples:

- Linked List Cycle II
- Remove Nth Node From End
- Palindrome Linked List
- Find the Duplicate Number

Harder Fast-Slow / Functional Graph Problems

Target:

40–60 minutes

Examples:

- Circular Array Loop
 - Reorder List
 - Sort List splitting
 - Reverse Nodes in k-Group support logic
-

31. Expanded Practice Set

31.1 Must-Do Easy

- 30. Middle of the Linked List
- 31. Linked List Cycle
- 32. Happy Number

31.2 Must-Do Medium

- 33. Linked List Cycle II
- 34. Remove Nth Node From End
- 35. Palindrome Linked List
- 36. Find the Duplicate Number
- 37. Reorder List
- 38. Split Linked List in Parts

31.3 Strong Interview Practice

- 39. Circular Array Loop
- 40. Sort List
- 41. Reverse Nodes in k-Group
- 42. Intersection of Two Linked Lists
- 43. Delete the Middle Node of a Linked List
- 44. Swapping Nodes in a Linked List
- 45. Maximum Twin Sum of a Linked List

31.4 Hard / Stretch

- 46. Circular Array Loop
 - 47. Find the Duplicate Number
 - 48. Reverse Nodes in k-Group
 - 49. Sort List
 - 50. Functional graph cycle detection variants
-

32. Suggested Study Order

Step 1 — Basic Speed Difference

Start with:

- 51. Middle of the Linked List
- 52. Linked List Cycle

Goal:

Understand fast moving twice as fast.

Step 2 — Cycle Entry

Solve:

- 53. Linked List Cycle II
- 54. Happy Number
- 55. Find the Duplicate Number

Goal:

Understand cycle entry logic beyond simple detection.

Step 3 — Gap Pointers

Solve:

- 56. Remove Nth Node From End
- 57. Kth Node From End

Goal:

Understand fixed-distance pointers.

Step 4 — Combined Linked List Problems

Solve:

- 58. Palindrome Linked List
- 59. Reorder List
- 60. Delete the Middle Node

Goal:

Combine middle finding with reversal and reconnection.

Step 5 — Advanced Functional Graph

Solve:

- 61. Circular Array Loop

Goal:

Apply fast-slow outside ordinary linked lists.

33. Pattern Decision Table

find middle	slow 1 step, fast 2 steps	Middle of List
detect cycle	Floyd detection	Linked List Cycle
find cycle start	meeting + reset	Cycle II
number transformation loop	Floyd on function	Happy Number
array values point to indices	Floyd on value graph	Duplicate Number
nth from end	fixed gap	Remove Nth Node
palindrome list	middle + reverse second half	Palindrome List
reorder front/back	middle + reverse + merge	Reorder List

Problem Clue	Technique	Example
split list	first-middle variant	Sort List
circular index loop	fast-slow over indices	Circular Array Loop

34. Final Cheat Sheet

Middle

```

ListNode* slow = head;
ListNode* fast = head;

while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
}

return slow;

```

Cycle Detection

```

while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;

    if (slow == fast) {
        return true;
    }
}

return false;

```

Cycle Entry

```

ListNode* entry = head;

while (entry != slow) {
    entry = entry->next;
    slow = slow->next;
}

return entry;

```

Remove Nth From End

```

ListNode dummy(0);
dummy.next = head;

ListNode* fast = &dummy;
ListNode* slow = &dummy;

for (int i = 0; i < n; i++) {
    fast = fast->next;
}

while (fast->next != nullptr) {
    fast = fast->next;
    slow = slow->next;
}

slow->next = slow->next->next;

return dummy.next;

```

Find Duplicate Number

```

int slow = nums[0];
int fast = nums[0];

do {
    slow = nums[slow];
    fast = nums[nums[fast]];
} while (slow != fast);

int entry = nums[0];

while (entry != slow) {
    entry = nums[entry];
    slow = nums[slow];
}

return entry;

```

35. What to Master Before Chapter 18

Before moving to Recursion, make sure you can:

- Find middle using fast/slow
 - Detect linked list cycle
 - Find cycle entry
 - Explain Floyd's algorithm
 - Use pointer comparison, not value comparison
 - Remove nth node from end with gap
 - Apply cycle detection to Happy Number
 - Apply cycle detection to Find Duplicate Number
 - Reverse second half of linked list
 - Solve Palindrome Linked List
 - Solve Reorder List
 - Handle null checks safely
 - Know first-middle vs second-middle loop conditions
-

36. Core Takeaway

Fast and Slow Pointers use different speeds to reveal hidden structure.
The main ideas are:

fast moves twice as fast -> slow reaches middle
fast catches slow in cycle -> cycle exists
reset one pointer to head -> find cycle entry
fixed gap -> find nth node from end

This pattern is especially powerful because it often gives:

$O(n)$ time
 $O(1)$ space

without extra data structures.

Fast-slow pointers are most common in linked lists, but the deeper idea applies to any system where each state points to a next state.

That includes:

- linked lists
- number transformations
- arrays as functional graphs
- circular index movement

The key question is:

If I repeatedly follow next pointers, can I enter a cycle or use speed difference to find a special position?
